SAM Image Studio

For Developer

Version 1.4.2

Local plugin API, complete source access, a tiled TorchScript U-Net example and reproducible validation commands.

Revised from the original v1.4 release
October 22, 2026

Contents

Scope, plugin contract and source access 3

Minimal executable plugin 4

Processor context and parameter specification 5

GUI actions, all functions and transactional edits 6

U-Net example: files and model contract 7

U-Net example: exporting and testing 9

GPU development in Conda 10

Application regression checks and genuine assets 11

Failure handling, versioning and references 12

Calibration, restoration and SAM3.1 extension points 13

Scope, plugin contract and source access

This manual is for plugin authors. User operations are documented separately. Plugin API v1 loads ordinary trusted local Python; plugins have the permissions of the Python process and are not sandboxed. Install only code you intend to run. No credentials or remote access are required by the plugin API.

A plugin is a local folder with plugin.json and a Python module. Install local plugin folder copies it into the application profile. Manage / reload plugins enables, disables, reloads or removes installed plugin code. Checkpoints should stay outside that folder and be supplied as parameters. Source symlinks and folders over 512 MiB are rejected.

All application source ships in samstudio. api.window is the real MainWindow, so advanced GUI actions can access its functions, canvas, current document, histories and stores. Import samstudio modules to use lower-level functions. API v1 is stable for the documented processor contract; direct internal window/module calls may need changes in later versions.

Module	Available work
domain	Document, Annotation, Group, Calibration, History, ProjectStore, exact codecs and atomic saves
main_window / canvas	GUI workflow methods, selections, drawing, worker jobs and native painting
exports	The v1.4 exact COCO RLE, YOLO, masks, image exports and cleaning extensions
inference / exemplars	Model catalog, load/predict/grid, downloads, LaMa and whole-image exemplar retrieval
processing / image_processing	Masks, curves, spatial restoration, synthetic noise, retained crops and paste-back
transforms / savepoints	Rotation, coordinate calibration, scaling and exact annotation snapshots
statistics / palette	Defined measurements, summaries and lookup-table colors
plugin_api / plugin_ui	Registration, processor validation, GUI plugin menus and Apply/Undo integration
hardware	Read-only NVIDIA detection and Conda-only installation plans

Minimal executable plugin

Create a folder with the following files. Use a unique ID and API version 1. module is a relative path inside the folder; the loader rejects absolute paths and parent traversal.

```
{
  "id": "my.lab.processor",
  "name": "My image processor",
  "api_version": 1,
  "module": "plugin.py",
  "version": "1.0"
}
```

```
def register(api):
    api.add_image_processor(
        "Half intensity",
        lambda context: context.image // 2,
        parameters={}
    )
```

The example accepts integer images. For a general processor, explicitly preserve dtype and validate supported channel types. Install the folder using Plugin > Install local plugin folder. Its action appears in Plugin. The application runs it in a worker, shows the returned-image preview and applies it only after Apply.

Additional local modules can be imported relatively, for example from .helpers import denoise. Use Path(__file__).parent to locate plugin resources. Plugins may import installed Python packages; the manager does not automatically install dependencies or modify an environment.

Processor context and parameter specification

Member	Meaning / requirement
context.image	A private copy of input pixels, original dtype and channels. The worker may compute on this copy.
context.target	Boolean H x W target mask. Removed pixels/protected retained crop rectangles are excluded by the app.
context.parameters	Selected parameter values; defaults reload on dialog creation/method changes.
context.calibration	Deep-copied physical calibration dictionary, including coordinate basis/origin when present.
context.objects	Deep-copied annotation dictionaries, including exact packed masks, provenance and cleaning records.
context.progress(message)	Send a text progress message to the status bar.
context.cancel / check_cancelled()	A cancellation event and helper. Check between tiles or iterations.
return value	A NumPy image with exactly the input shape and dtype and finite values. Alpha is retained; pixels outside target are restored by the app.

```
api.add_image_processor("My denoiser", run, parameters={
    "weight": {"default": 0.08, "min": 0.001, "max": 1.0, "step": 0.01},
    "iterations": {"default": 100, "min": 1, "max": 1000, "step": 1},
    "checkpoint": {"default": ""},
    "use_option": {"default": True}
})
```

Boolean defaults create checkboxes, integers/floats create bounded spin controls, and strings create text fields. The callback must validate relations such as overlap < tile_size / 2. Use explicit numeric units. Avoid hidden normalization or silently reinterpreting physical calibration.

GUI actions, all functions and transactional edits

add_action registers a GUI-thread callback and passes its PluginAPI instance. It may open dialogs or call the real window. A background image processor must not read or mutate Qt widgets. Do not pass api.window into a worker for GUI calls.

```
def mark_reviewed(api):
    window = api.window
    doc = api.document
    if doc is None:
        return
    selected = list(window.selected_objects())
    def change():
        for obj in selected:
            obj.provenance["plugin_review_note"] = "Reviewed by my lab plugin"
    api.commit("Plugin review note", change)

def register(api):
    api.add_action("Set selected review notes", mark_reviewed)
```

api.commit uses History and the normal autosave path. A failed mutation restores the prior state. Undo/Redo and native project saving stay coordinated; exports use the current committed state. Mutating doc.objects directly outside a commit bypasses those guarantees and is an advanced internal operation.

Examples of source access: window.open_paths([...]), window.selected_objects(), window.export_dataset("coco"), window.run_inference(), window._start_job(...), Document.add_object(...), processing.shape_mask(...), exports.export_coco(...) and statistics.measurement_records(...). Inspect the actual callable signatures before using internals. Do not call GUI-export dialogs from a worker.

For deterministic processing, snapshot parameters and arrays before submitting a job. A plugin failure is reported without applying its result. The app checks output dimensions, dtype, finiteness, cancellation, outside-target pixels and alpha before an image result can be applied.

U-Net example: files and model contract

The release includes examples/plugins/unet_denoiser with a complete plugin, manifest and a tiny U-Net architecture used for interface tests. No trained denoising weights are bundled. A random-weight interface checkpoint is not a scientific denoiser and does not demonstrate restoration quality.

```
{"id":"example.unet_denoiser","name":"U-Net denoiser
    example","api_version":1,"module":"plugin.py","version":"1.0","description":"Tiled grayscale TorchScript
    U-Net; provide your trained checkpoint. No trained weights are bundled."}
```

The processor expects a grayscale TEM/SEM input. A TorchScript checkpoint takes [1, 1, tile, tile] floating tensors and returns the same shape. Its intended output is normalized to [0, 1]. The plugin clips to that range and converts back to input dtype. Adapt normalization explicitly if your trained model uses another range or detector-specific units.

Unsigned integers use their type maximum; floating/signed example inputs use their observed minimum/maximum. Tiles overlap and blend with a floored Hann window; borders are padded from real pixels. tile_size must be divisible by 4 and overlap must be less than half a tile. Every tile reports progress and checks cancellation.

```
"""API v1 example: a user-trained TorchScript U-Net, tiled with overlap blending."""

from pathlib import Path
import numpy as np


def denoise(context):
    import torch

    context.check_cancelled()
    p = context.parameters
    path = Path(p["checkpoint"]).expanduser()
    if not path.is_file():
        raise ValueError("Choose your trained TorchScript checkpoint file.")
    if context.image.ndim != 2:
        raise ValueError("This example expects a grayscale TEM/SEM image.")
    device = p.get("device", "cpu")
    if device.startswith("cuda"):
        import os

        if not os.environ.get("CONDA_PREFIX"):
            raise ValueError("GPU plugins must run inside Conda.")
        if not torch.cuda.is_available():
            raise ValueError("CUDA is unavailable.")
    model = torch.jit.load(str(path), map_location=device).eval()
    a = context.image
    low, high = (
        (0.0, float(np.iinfo(a.dtype).max))
        if a.dtype.kind == "u"
        else (float(a.min()), float(a.max()))
    )
    span = high - low or 1.0
    unit = (a.astype(np.float32) - low) / span
    tile = int(p["tile_size"])
    overlap = int(p["overlap"])
    if tile < 16 or tile % 4 or not 0 <= overlap < tile // 2:
        raise ValueError(
            "tile_size must be divisible by 4 and overlap must be less than half a tile."
        )
    stride = tile - overlap
    h, w = a.shape
    ph = max(tile, ((h - tile + stride - 1) // stride) * stride + tile)
    pw = max(tile, ((w - tile + stride - 1) // stride) * stride + tile)
    padded = np.pad(
        unit, ((0, ph - h), (0, pw - w)), mode="reflect" if min(h, w) > 1 else "edge"
    )
```

```
    output = np.zeros_like(padded, dtype=np.float64)
    weights = np.zeros_like(output)
    blend = np.maximum(np.outer(np.hanning(tile), np.hanning(tile)), 1e-3)
    tiles = [
        (y, x)
        for y in range(0, ph - tile + 1, stride)
        for x in range(0, pw - tile + 1, stride)
    ]
    with torch.inference_mode():
        for i, (y, x) in enumerate(tiles):
            context.check_cancelled()
            context.progress(f"U-Net tile {i+1}/{len(tiles)}")
            tensor = torch.from_numpy(padded[y : y + tile, x : x + tile].copy())[
                None, None
            ].to(device)
            result = model(tensor)
            if result.shape != tensor.shape:
                raise ValueError("The model must return [1,1,tile,tile].")
            value = result[0, 0].detach().float().cpu().numpy()
            if not np.isfinite(value).all():
                raise ValueError("The model returned nonfinite pixels.")
            output[y : y + tile, x : x + tile] += value * blend
            weights[y : y + tile, x : x + tile] += blend
    restored = np.clip(output[:h, :w] / weights[:h, :w], 0, 1) * span + low
    if a.dtype.kind in "iu":
        restored = np.rint(restored)
    return restored.astype(a.dtype)


def register(api):
    api.add_image_processor(
        "U-Net denoise",
        denoise,
        {
            "checkpoint": {"default": ""},
            "device": {"default": "cpu"},
            "tile_size": {"default": 256, "min": 16, "max": 2048, "step": 4},
            "overlap": {"default": 32, "min": 0, "max": 512, "step": 1},
        },
    )
```

U-Net example: exporting and testing

After training your own model, export it as TorchScript and provide its local checkpoint path in the plugin dialog. For this example, use a model whose supported shape contract is compatible with the selected tile size. Verify the trained-model normalization and restore it consistently on output.

```
"""Export a tiny U-Net for an interface test; random weights are NOT a denoiser."""

import argparse
import torch
from torch import nn


class TinyUNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(8, 8, 3, padding=1),
            nn.ReLU(),
        )
        self.pool = nn.AvgPool2d(2)
        self.mid = nn.Sequential(nn.Conv2d(8, 16, 3, padding=1), nn.ReLU())
        self.up = nn.ConvTranspose2d(16, 8, 2, stride=2)
        self.out = nn.Conv2d(16, 1, 1)

    def forward(self, x):
        skip = self.enc(x)
        b = self.up(self.mid(self.pool(skip)))
        return torch.sigmoid(self.out(torch.cat((skip, b), dim=1)))


if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("output")
    args = p.parse_args()
    torch.manual_seed(7)
    torch.jit.script(TinyUNet().eval()).save(args.output)
    print(
        "Interface-test checkpoint only. Train a model before using it for denoising."
    )
```

Interface smoke test: activate the CPU environment, run the following commands, install the example plugin folder, choose U-Net denoise and enter the resulting checkpoint path. This verifies model invocation, tiling, dtype and GUI integration only. Use trained weights for actual denoising.

```
python examples/plugins/unet_denoiser/model_example.py /local/path/untrained_test.pt
python -m pytest -q tests/test_v141_segmentation_plugins_gpu.py -k unet
```

The automated test scripts the actual TinyUNet, loads it through torch.jit.load and processes a non-tile-sized uint16 image. It checks complete output shape/type, changed target pixels and exact outside-target preservation. Additional tests reject malformed outputs, nonfinite values, registration errors and cancelled jobs.

For restoration validation, use independent paired clean/noisy data or a justified self-supervised protocol; record dose, pixel size, detector normalization and train/test separation. Compare MSE/PSNR/SSIM where meaningful, and evaluate morphology/edge bias against acquired pixels. Passing interface tests does not establish scientific accuracy.

GPU development in Conda

GPU execution is restricted to a dedicated Conda environment. The example U-Net rejects CUDA when CONDA_PREFIX is absent. The main SAM loader enforces the same rule. CPU remains available without Conda. Avoid changing the system Python or the default CPU environment to add CUDA.

```
python scripts/install_conda_gpu.py --inspect
python scripts/install_conda_gpu.py --name sam-image-studio-gpu --dry-run
python scripts/install_conda_gpu.py --name sam-image-studio-gpu
conda activate sam-image-studio-gpu
python -m samstudio
```

Native SAM3 is optional and requires approved checkpoint access. The supplied native SAM3 dependency recipe targets Linux/WSL2; select --sam3 there. The official native project lists CUDA prerequisites. This app does not enable an unverified SAM3 CPU path.

Inspect GPU detection separately from model acceptance. Device identification, driver-capability selection and Conda command planning are tested with explicit synthetic reports. The release machine has no NVIDIA device, so CUDA kernels, GPU memory use and full SAM3 weights are not claimed as tested. Run the application self-test and real model previews on each target environment.

Plugin dependencies should be installed manually into the active environment with pinned versions and pip check. Keep plugin requirements next to source and document them. Do not automatically install packages from a register callback.

Application regression checks and genuine assets

```
python -m pip install -r requirements-dev.txt
python -m pytest -q tests/test_v141_processing_workspace.py tests/test_v141_segmentation_plugins_gpu.py
python scripts/fetch_validation_assets.py --models --text --directory /local/validation_assets
python scripts/run_full_validation.py --assets /local/validation_assets --output
    validation/local_full_results.xml
```

The complete runner requires actual SAM1, SAM2.1, LaMa, public microscopy reader fixtures and pinned Grounding DINO assets. It rejects skipped tests. A plain pytest run without external assets may skip genuine-model integrations. New genuine exemplar tests verify matching outside a positive shape, an excluded negative instance and actual pooled SAM features.

The native one-object regression tests explicitly choose Single object, because whole-image exemplars now have a distinct intended behavior. GUI tests cover v1.4 scope retention, saved annotations during workspace removal, startup decisions, processing/plugin worker previews with Apply/Cancel and Undo, and automatic-grid/text dispatch.

Before distributing a changed build: run meaningful tests, build a wheel, verify source/wheel/installed code equality, run installed-package checks outside the source directory, include PDF resources, inspect GUI/PDF rendering and record hardware/weight hashes. Never report simulated GPU setup as genuine CUDA or model validation.

Failure handling, versioning and references

Use a new plugin version for each behavior change. Keep api_version at 1 only while adhering to this contract. Registration errors leave the plugin unavailable but preserve the app. Remove/reinstall a plugin folder to replace its code, then reload it. Store checkpoints separately so source updates do not duplicate large models.

Native schema 5 reads earlier schemas 1-4. This revision extends v1.4 native recovery with retained crop snapshots and exemplar prompts. Annotation data, COCO export and existing property dialogs retain the v1.4 contract. Keep native projects for complete editable recovery.

https://pytorch.org/docs/stable/jit.html

https://github.com/facebookresearch/sam3

https://scikit-image.org/docs/stable/api/skimage.restoration.html

https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html

Calibration, restoration and SAM3.1 extension points

calibration_table.py reads extension-insensitive converted_image_name/converted_size workbooks and applies calibration independently of COCO editing. apply_calibration reconstructs the basis/origin through saved affine rotations. It does not change mask geometry. The Statistics menu imports a lookup transactionally; later opens apply it after native restore.

denoise_config.py is the shared public control schema: default, min/max/step or choices, help and recommended. Its descriptions feed both the GUI and parameter guide. image_processing.parameters validates values and cross-field constraints before launching the worker. process_pixels normalizes intensity, dispatches independent spatial channels, preserves alpha, converts back to the original dtype and composites only target pixels. The existing plugin parameter contract remains compatible with four-item numeric tuples and dictionary specifications.

compressed_sensing.py implements sampled reconstruction and relaxed-fidelity denoising. sparse_denoising.py calls SPORCO and official bm3d. neural_denoising.py uses CAREamics 0.3.5 factories for N2V/N2V2, trains/saves per channel, then predicts using tiled native CAREamics inference. N2V2 is the backend's actual use_n2v2 configuration, not a renamed ordinary U-Net. Training uses no batch normalization, mean/std backend normalization, no multiprocessing workers and no experiment logger. No external cloud logging is configured. Dependency upgrades require rerunning real train/save/reload tests for both variants.

All image previews are transactional. Model-training files are intentional side effects before Apply and are not rolled back by image Undo. Workers check cancellation between iterations/batches; blocking library calls finish before cancellation is delivered. Keep GUI widgets on the GUI thread. initialize_torch_runtime primes native thread pools before Qt workers run.

sam31_backend.py adapts the official build_sam3_multiplex_video_predictor API. A temporary lossless PNG is used as a single-frame resource; each native session closes in finally and the PNG is removed. Keep all semantic positive/negative boxes in one add_prompt request. Video predictor geometry is normalized TOP-LEFT XYWH; SAM3 image processor geometry remains CENTER XYWH. Mixing these conventions shifts prompts. Point exemplars first invoke an interactive instance session, then supply its box to the concept session. Requested CUDA device contexts surround construction and requests. use_fa3, compilation and warm-up are disabled in this integration. Actual hardware/approved weights must pass local acceptance; API contract tests are not inference validation.

Add a backend by extending the method schema, dispatcher, optional dependency handling and real numerical tests. A meaningful test compares against an independent algorithm reference or a known-input result; do not substitute a stub for a claimed scientific backend. test_v142_calibration_denoise.py covers notebook-equivalent reconstruction, full-sampling identity versus actual denoising, synthetic Gaussian error, actual SPORCO/BM3D and CAREamics train/save/reload. One-epoch N2V checks prove the interface, not scientific denoising quality. test_v142_sam31.py checks native requests/geometry/cleanup without claiming CUDA execution.